Walmart Black Friday: Do Women Spend More Than Men?

Confidence Intervals and the Central Limit Theorem applied to 550,068 transactions

Author: Guntuku Hariprasad | **Program:** Scaler, Business Case Study | **Tools:** Python, pandas, NumPy, SciPy, Matplotlib, Seaborn

---

Executive summary

Question	Answer (95% confidence, per transaction)
Do women spend more per transaction than men?	**No.** Men average **9,437.5** vs women **8,734.6**, a gap of about **703 (8%)**.
Do the male and female confidence intervals overlap?	**No.** Female [8,709, 8,760] and Male [9,422, 9,453] are far apart at 90%, 95% and 99%.
Does marital status change spend per transaction?	**No.** Unmarried 9,265.9 vs Married 9,261.2; the intervals overlap almost completely.
Does age change spend per transaction?	**Yes, modestly.** Spend rises with age: 0-17 (8,934) < 18-25 (9,170) < 26-35 (9,253) ≈ 36-50 (9,295) < 51+ (9,464).
What drives the gender gap?	**Basket mix, not price sensitivity.** Men buy more of high-ticket Category 1; women buy more of lower-ticket Categories 5 and 8. Within the same product category, women spend just as much as men; the entire gap is explained by product mix.
Where is the revenue?	Men are 75% of transactions and 77% of Black Friday revenue; customers aged 26-35 are the single most valuable group.

Bottom line for Walmart: men and women behave differently in *what* they buy, not just *how much*. The biggest lever is to move women into higher-value categories and to win more female shoppers, while keeping the 26-35 core and the high-ticket-friendly 51+ segment engaged.

Table of contents

1. Problem statement and approach
2. Data loading, structure and basic metrics
3. Non-graphical analysis: value counts and unique values
4. Missing values and outlier detection
5. Visual analysis: univariate, bivariate and correlation
6. Gender: are women spending more per transaction than men?
7. Gender: Central Limit Theorem and confidence intervals (90 / 95 / 99%)
8. Marital status: Married vs Unmarried
9. Age: life-stage bins (0-17, 18-25, 26-35, 36-50, 51+)
10. Bonus: the customer-level view (total spend per shopper)
11. Final insights
12. Recommendations for Walmart

1. Problem statement and approach

Business problem. Walmart's management wants to understand how purchase amount varies with customer gender and other attributes, and specifically: *do women spend more on Black Friday than men?* We assume the population is **50 million male and 50 million female customers**, and the Black Friday transactions in this dataset are a sample from that population.

Why statistics is needed. The data is a sample, so any average we compute is an *estimate*. A different sample would give a slightly different number. A **confidence interval (CI)** gives a range that, with a chosen level of confidence (90%, 95% or 99%), contains the true population average. The **Central Limit Theorem (CLT)** tells us that the average of a reasonably large sample is approximately normally distributed, even when individual purchases are not, which is what lets us build those intervals.

Approach.

1. Profile the data: shape, types, summary statistics, missing values, outliers.
2. Explore it visually to understand distributions and relationships.

3. For each segment (Gender, Marital Status, Age): bootstrap sample means at several sample sizes to watch the CLT at work, then compute 90/95/99% CIs and check whether the intervals overlap.
4. Translate the results into plain-language recommendations.

Unit of analysis. The case asks about spend **per transaction**, so the main analysis uses each row's `Purchase`. Section 10 repeats the key test on **total spend per customer** as a robustness check.

2. Data loading, structure and basic metrics

```
import warnings, os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

warnings.filterwarnings("ignore")
pd.set_option("display.float_format", lambda v: f"{v:,.2f}")
pd.set_option("display.width", 140)

# Consistent, colour-blind-checked palette
BLUE, ORANGE, AQUA, VIOLET, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#6250d6", "#8a8984"
GENDER_PAL  = {"Female": ORANGE, "Male": BLUE}
MARITAL_PAL = {"Unmarried": AQUA, "Married": VIOLET}
AGE_ORDER   = ["0-17", "18-25", "26-35", "36-50", "51+"]
AGE_PAL     = dict(zip(AGE_ORDER, ["#b7d3f2", "#7fb0e8", "#2a78d6", "#1d58a3", "#103a70"]))

sns.set_theme(style="whitegrid", context="notebook", font_scale=0.95)
plt.rcParams.update({"figure.dpi": 90, "axes.spines.top": False, "axes.spines.right": False,
                     "axes.titleweight": "bold", "axes.titlesize": 11, "grid.color": "#e6e5e0"})

IMG_DIR = "../images"
os.makedirs(IMG_DIR, exist_ok=True)
def save(fig, name):
    fig.savefig(f"{IMG_DIR}/{name}.png", dpi=130, bbox_inches="tight")

RNG = np.random.default_rng(42)   # reproducible sampling
```

```
df = pd.read_csv("../data/walmart_data.csv")
print("Shape:", df.shape)
df.head()
```

```
Shape: (550068, 10)
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   User_ID                     550068 non-null  int64
 1   Product_ID                  550068 non-null  str
 2   Gender                      550068 non-null  str
 3   Age                         550068 non-null  str
 4   Occupation                  550068 non-null  int64
 5   City_Category               550068 non-null  str
 6   Stay_In_Current_City_Years  550068 non-null  str
 7   Marital_Status              550068 non-null  int64
 8   Product_Category            550068 non-null  int64
 9   Purchase                    550068 non-null  int64
dtypes: int64(5), str(5)
memory usage: 42.0 MB
```

Observations on structure

- **550,068 rows × 10 columns.** Each row is one product purchased by one customer on Black Friday.
- `User_ID`, `Occupation`, `Marital_Status` and `Product_Category` are stored as integers, but they are **codes / labels**, not quantities. Averaging them is meaningless, so they are converted to `category`.
- `Purchase` is the only true numeric (continuous) variable.
- Converting the 9 descriptive columns to `category` cuts memory sharply and makes intent explicit.

```
mem_before = df.memory_usage(deep=True).sum() / 1024**2
cat_cols = ["User_ID", "Product_ID", "Gender", "Age", "Occupation", "City_Category",
            "Stay_In_Current_City_Years", "Marital_Status", "Product_Category"]
df[cat_cols] = df[cat_cols].astype("category")

# Readable labels for reporting
df["Gender"] = df["Gender"].cat.rename_categories({"F": "Female", "M": "Male"})
df["Marital_Status"] = df["Marital_Status"].cat.rename_categories({0: "Unmarried", 1: "Married"})

# Life-stage age bins requested in the case (original bins merged)
life_stage = {"0-17": "0-17", "18-25": "18-25", "26-35": "26-35", "36-45": "36-50",
              "46-50": "36-50", "51-55": "51+", "55+": "51+"}
df["Life_Stage"] = pd.Categorical(df["Age"].map(life_stage), categories=AGE_ORDER, ordered=True)
df["Age"] = df["Age"].cat.reorder_categories(["0-17","18-25","26-35","36-45","46-50","51-55","55+"],
ordered=True)

mem_after = df.memory_usage(deep=True).sum() / 1024**2
print(f"Memory: {mem_before:,.1f} MB -> {mem_after:,.1f} MB")
df.dtypes
```

```
Memory: 158.4 MB -> 10.7 MB
User_ID                     category
Product_ID                  category
Gender                      category
Age                         category
Occupation                  category
City_Category               category
Stay_In_Current_City_Years  category
Marital_Status              category
Product_Category            category
Purchase                       int64
Life_Stage                  category
dtype: object
```

```
df.describe(include="all").T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
User_ID	550,068.00	5,891.00	1,001,680.00	1,026.00	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_ID	550068	3631	P00265242	1880	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Gender	550068	2	Male	414259	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Age	550068	7	26-35	219587	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Occupation	550,068.00	21.00	4.00	72,308.00	NaN	NaN	NaN	NaN	NaN	NaN	NaN
City_Category	550068	3	B	231173	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Stay_In_Current_City_Years	550068	5	1	193821	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Marital_Status	550068	2	Unmarried	324731	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Product_Category	550,068.00	20.00	5.00	150,933.00	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Purchase	550,068.00	NaN	NaN	NaN	9,263.97	5,023.07	12.00	5,823.00	8,047.00	12,054.00	23,961.00
Life_Stage	550068	5	26-35	219587	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
p = df["Purchase"]
summary = pd.Series({"mean": p.mean(), "median": p.median(), "mean - median": p.mean() - p.median(),
                     "std": p.std(), "min": p.min(), "max": p.max(),
                     "skewness": p.skew(), "kurtosis": p.kurt()})
summary.to_frame("Purchase")
```

	Purchase
mean	9,263.97
median	8,047.00
mean - median	1,216.97
std	5,023.07
min	12.00
max	23,961.00
skewness	0.60
kurtosis	-0.34

Statistical summary: observations

- **Purchase ranges from 12 to 23,961**; the average transaction is **9,264** and the median is **8,047**.
- **Mean > median by ~1,217**, and skewness is **+0.60**, so the distribution is **right-skewed**: a minority of large purchases pulls the average up.
- **5,891 unique customers** bought **3,631 unique products**: on average each customer made ~93 transactions, so the data is rich enough for stable estimates.
- Most frequent values: Male, age 26-35, City B, 1 year in current city, Unmarried, Product Category 5.

3. Non-graphical analysis: value counts and unique values

```
df[cat_cols + ["Life_Stage"]].nunique().to_frame("unique values").T
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Life_Stage
unique values	5891	3631	2	7	21	3	5	2	20	5

```
def vc(col):
    t = df[col].value_counts()
    return pd.DataFrame({"count": t, "share %": (t / len(df) * 100).round(2)})

for col in ["Gender", "Life_Stage", "Age", "City_Category", "Stay_In_Current_City_Years", "Marital_Status"]:
    print(f"\n--- {col} ---")
    print(vc(col).sort_index().to_string())
```

```
--- Gender ---
         count  share %
Gender
Female  135809    24.69
Male    414259    75.31

--- Life_Stage ---
            count  share %
Life_Stage
0-17        15102     2.75
18-25       99660    18.12
26-35      219587    39.92
36-50      155714    28.31
51+         60005    10.91

--- Age ---
        count  share %
Age
0-17    15102     2.75
18-25   99660    18.12
26-35  219587    39.92
36-45  110013    20.00
46-50   45701     8.31
51-55   38501     7.00
55+     21504     3.91

--- City_Category ---
                count  share %
City_Category
A              147720    26.85
B              231173    42.03
C              171175    31.12

--- Stay_In_Current_City_Years ---
                           count  share %
Stay_In_Current_City_Years
0                          74398    13.53
1                         193821    35.24
2                         101838    18.51
3                          95285    17.32
4+                         84726    15.40

--- Marital_Status ---
                count  share %
Marital_Status
Unmarried      324731    59.03
Married        225337    40.97
```

```python
users = (df.groupby("User_ID", observed=True)
           .agg(Gender=("Gender", "first"), Marital_Status=("Marital_Status", "first"),
                Life_Stage=("Life_Stage", "first"), City=("City_Category", "first"),
                Total_Spend=("Purchase", "sum"), Transactions=("Purchase", "size"),
                Avg_Ticket=("Purchase", "mean"))
           .reset_index())
print("Unique customers by gender:")
print(users["Gender"].value_counts().to_string())
print("\nTransactions per customer, by gender:")
users.groupby("Gender", observed=True)[["Transactions", "Total_Spend", "Avg_Ticket"]].mean()
```

```
Unique customers by gender:
Gender
Male      4225
Female    1666

Transactions per customer, by gender:
```

Gender	Transactions	Total_Spend	Avg_Ticket
Female	81.52	712,024.39	8,965.20
Male	98.05	925,344.40	9,806.87

Observations

- **Gender:** 75.3% of transactions are by men and 24.7% by women. By people, **4,225 men vs 1,666 women (72% / 28%)**. Men also transact more often (~98 vs ~82 items each).

- **Age:** 26-35 is the largest group (39.9% of transactions); 0-17 is the smallest (2.7%).
- **City:** City B has the most transactions (42%), then C (31%) and A (27%).
- **Stay in city:** 1 year is most common (35%); newcomers (0 years) are the smallest group.
- **Marital status:** 59% unmarried, 41% married.
- **Product category:** 20 categories; 1, 5 and 8 account for ~74% of all transactions.
- The **sample is not 50/50 by gender**, while the stated population is. That is exactly why we estimate *averages* with confidence intervals per group, rather than comparing raw totals.

4. Missing values and outlier detection

```
missing = df.isnull().sum()
print("Missing values per column:\n", missing.to_string())
print("\nTotal missing:", missing.sum(), "| Duplicate rows:", df.duplicated().sum())
```

```
Missing values per column:
 User_ID                      0
Product_ID                    0
Gender                        0
Age                           0
Occupation                    0
City_Category                 0
Stay_In_Current_City_Years    0
Marital_Status                0
Product_Category              0
Purchase                      0
Life_Stage                    0
Total missing: 0 | Duplicate rows: 0
```

```
q1, q3 = p.quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df[(p < lower) | (p > upper)]
print(f"Q1 = {q1:,.0f} | Q3 = {q3:,.0f} | IQR = {iqr:,.0f}")
print(f"Whiskers: [{lower:,.0f}, {upper:,.0f}]")
print(f"Outliers: {len(outliers):,} rows ({len(outliers)/len(df):.2%}), all on the high side (>
{upper:,.0f})")
print("\nOutliers by gender (% of that gender's transactions):")
print((outliers["Gender"].value_counts() / df["Gender"].value_counts() * 100).round(2).to_string())
print("\nTop product categories among outliers:")
print(outliers["Product_Category"].value_counts().loc[lambda s: s > 0].to_string())
```

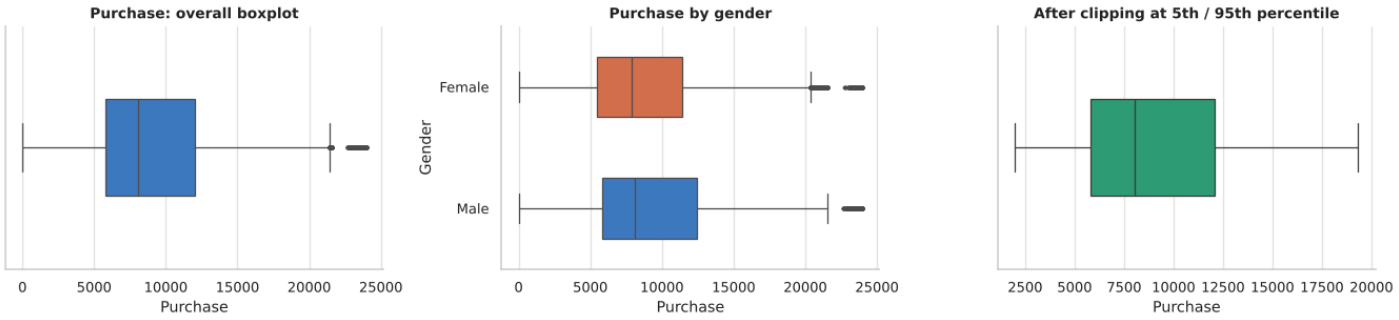
```
Q1 = 5,823 | Q3 = 12,054 | IQR = 6,231
Whiskers: [-3,524, 21,400]
Outliers: 2,677 rows (0.49%), all on the high side (>21,400)

Outliers by gender (% of that gender's transactions):
Gender
Male     0.50
Female   0.43

Top product categories among outliers:
Product_Category
10    2275
15     327
9       75
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6))
sns.boxplot(x=p, ax=axes[0], color=BLUE, width=0.4, fliersize=2)
axes[0].set_title("Purchase: overall boxplot")
sns.boxplot(data=df, x="Purchase", y="Gender", palette=GENDER_PAL, ax=axes[1], width=0.5, fliersize=2)
axes[1].set_title("Purchase by gender")
clipped = p.clip(p.quantile(0.05), p.quantile(0.95))
sns.boxplot(x=clipped, ax=axes[2], color=AQUA, width=0.4)
axes[2].set_title("After clipping at 5th / 95th percentile")
plt.tight_layout(); save(fig, "01_outliers_boxplots"); plt.show()

print(f"Mean before clipping: {p.mean():,.1f} | after: {clipped.mean():,.1f} | median: {p.median():,.0f}")
```




```
Mean before clipping: 9,264.0 | after: 9,256.7 | median: 8,047
```

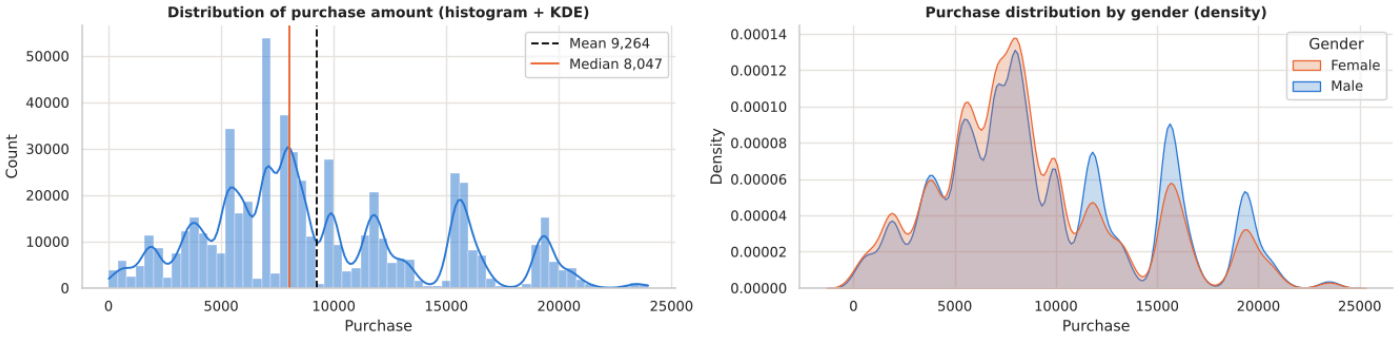
Missing values and outliers: observations

- **No missing values and no duplicate rows**, so no imputation is required.
- The IQR rule flags **2,677 transactions (0.49%)** above **21,400**, all on the high side. They come almost entirely from high-ticket product categories (10, 15 and 9; Category 10 alone is 85%), so they are **genuine large purchases, not data errors**.
- Clipping at the 5th/95th percentile moves the mean by only ~7 (9,264 → 9,257). Because the outliers are real and few, and because the CLT works on *sample means* (which are robust here), **we keep all rows** for the confidence-interval analysis. Removing real high spenders would bias the estimate of what customers actually spend.

5. Visual analysis

5.1 Univariate: the continuous variable (`Purchase`)

```
fig, axes = plt.subplots(1, 2, figsize=(15, 3.8))
sns.histplot(p, bins=60, kde=True, color=BLUE, ax=axes[0], edgecolor="white", linewidth=0.3)
axes[0].axvline(p.mean(), color="#0b0b0b", ls="--", lw=1.5, label=f"Mean {p.mean():,.0f}")
axes[0].axvline(p.median(), color=ORANGE, ls="-", lw=1.5, label=f"Median {p.median():,.0f}")
axes[0].legend(); axes[0].set_title("Distribution of purchase amount (histogram + KDE)")
sns.kdeplot(data=df, x="Purchase", hue="Gender", palette=GENDER_PAL, common_norm=False, fill=True,
alpha=0.25, ax=axes[1])
axes[1].set_title("Purchase distribution by gender (density)")
plt.tight_layout(); save(fig, "02_purchase_distribution"); plt.show()
```

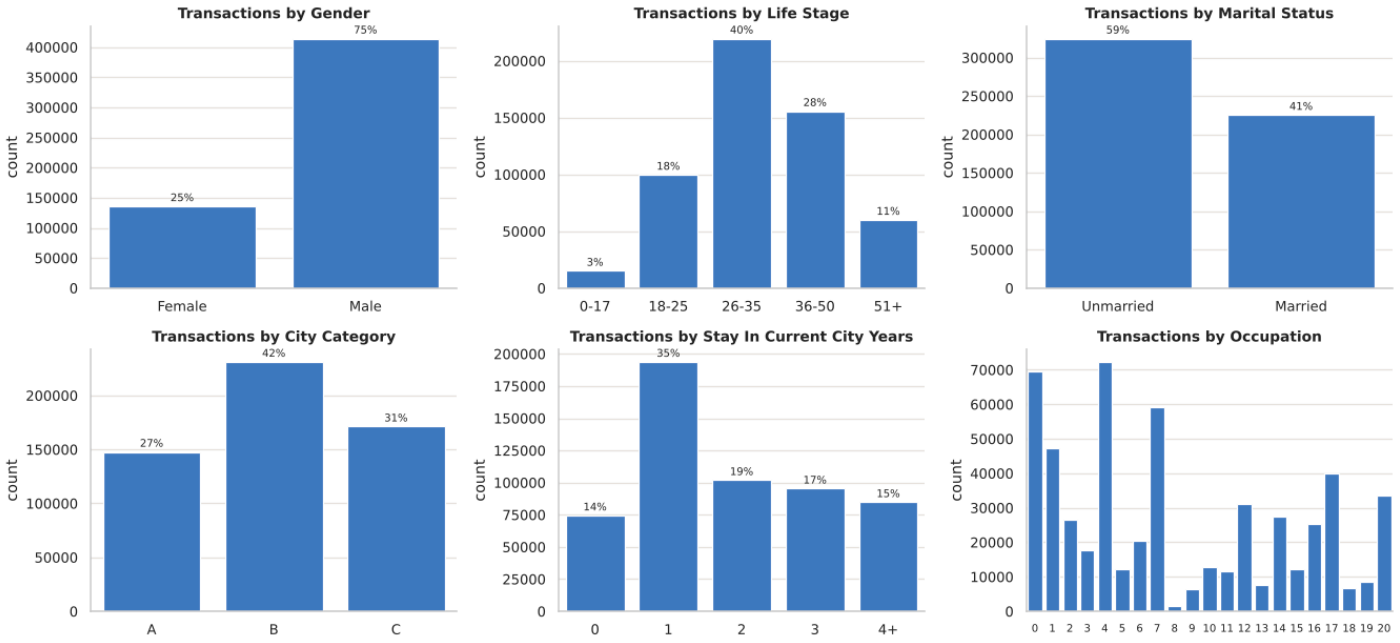


Observation: Purchase is **not normal**. It is right-skewed and **multi-modal**, with peaks near ~6-8K, ~11K, ~15-16K and ~19-20K. These peaks line up with the price bands of different product categories, so the distribution is really a *mixture* of category price levels. The male curve has more mass in the 15K-20K range, the female curve more in the 5K-8K range. Because individual purchases are far from normal, we will rely on the **CLT** (distribution of *sample means*) rather than on the raw distribution.

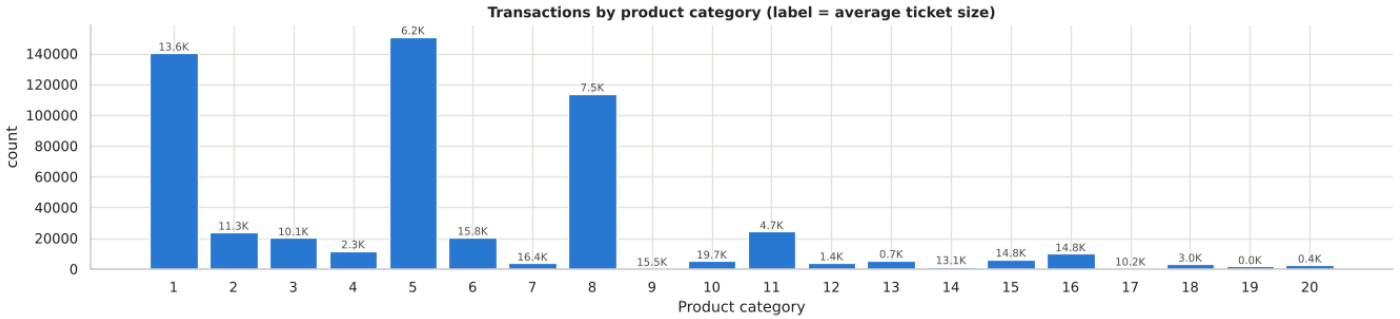
5.2 Univariate: categorical variables (count plots)

```
cols = ["Gender", "Life_Stage", "Marital_Status", "City_Category", "Stay_In_Current_City_Years",
"Occupation"]
fig, axes = plt.subplots(2, 3, figsize=(15, 7))
for ax, col in zip(axes.flat, cols):
    order = df[col].cat.categories
    sns.countplot(data=df, x=col, order=order, color=BLUE, ax=ax, edgecolor="white")
    ax.set_title(f"Transactions by {col.replace('_', ' ')}"); ax.set_xlabel(""); ax.set_ylabel("count")
    if col != "Occupation":
        for c in ax.containers:
            ax.bar_label(c, labels=[f"{v/len(df):.0%}" for v in c.datavalues], fontsize=8, padding=2)
    else:
```

```
        ax.tick_params(axis="x", labelsize=8)
plt.tight_layout(); save(fig, "03_countplots_categorical"); plt.show()
```




```
fig, ax = plt.subplots(figsize=(15, 3.6))
pc = df["Product_Category"].value_counts().sort_index()
ax.bar(pc.index.astype(str), pc.values, color=BLUE, edgecolor="white")
ax2_vals = df.groupby("Product_Category", observed=True)["Purchase"].mean()
for x, (cnt, avg) in enumerate(zip(pc.values, ax2_vals.reindex(pc.index).values)):
    ax.text(x, cnt, f"{avg/1000:.1f}K", ha="center", va="bottom", fontsize=8, color="#52514e")
ax.set_title("Transactions by product category (label = average ticket size)")
ax.set_xlabel("Product category"); ax.set_ylabel("count")
plt.tight_layout(); save(fig, "04_product_category"); plt.show()
```



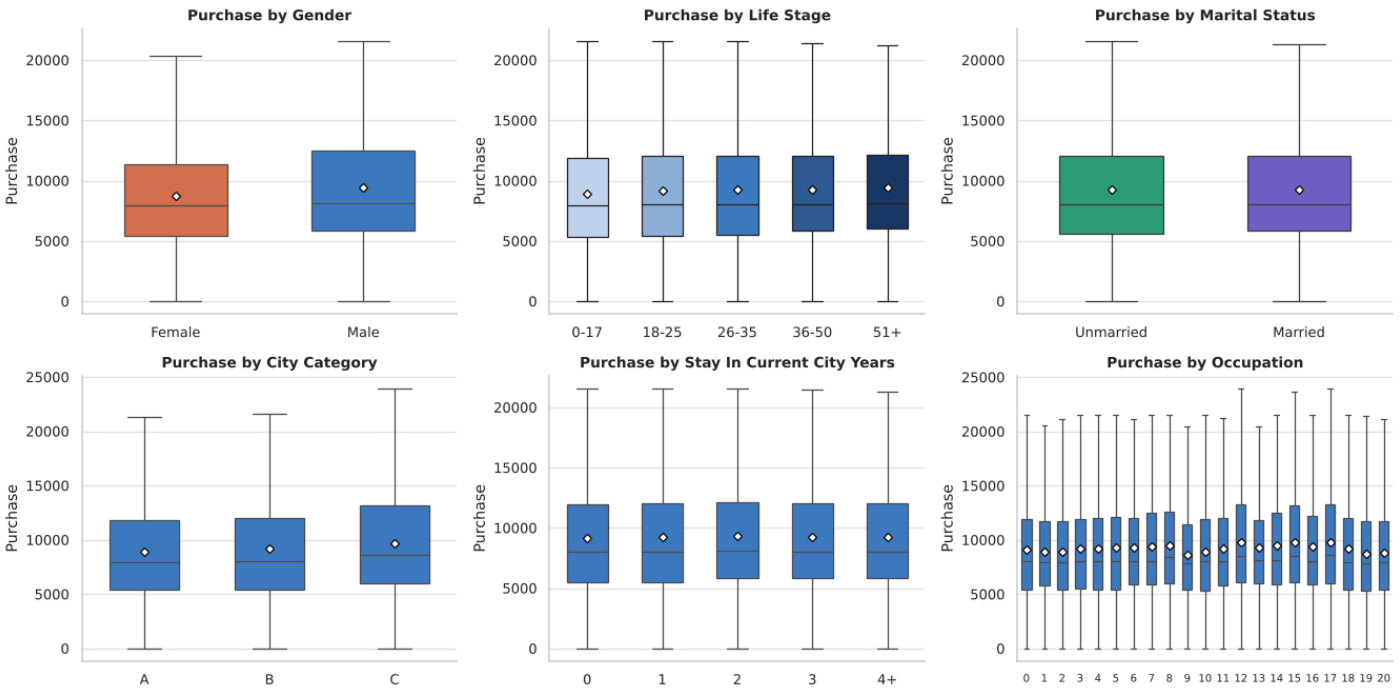
Observations (univariate)

- **Men dominate transaction volume (75%)**, so total revenue comparisons would be misleading. We must compare *averages*.
- **26-35 is the core shopper (40%)**, followed by 36-50 (28%). Teenagers (0-17) are only 3%.
- **Unmarried shoppers (59%)** outnumber married ones.
- **City B** leads in volume; **1-year residents** are the largest tenure group, suggesting recent movers are active shoppers.
- **Occupations 4, 0 and 7** are the most frequent; 8 is rare.
- **Product categories 5, 1 and 8 drive ~74% of transactions**, but their ticket sizes differ hugely: Category 1 averages **13.6K**, Category 5 **6.2K**, Category 8 **7.5K**. High-ticket categories such as 10 (19.7K), 7, 6 and 9 are low-volume.

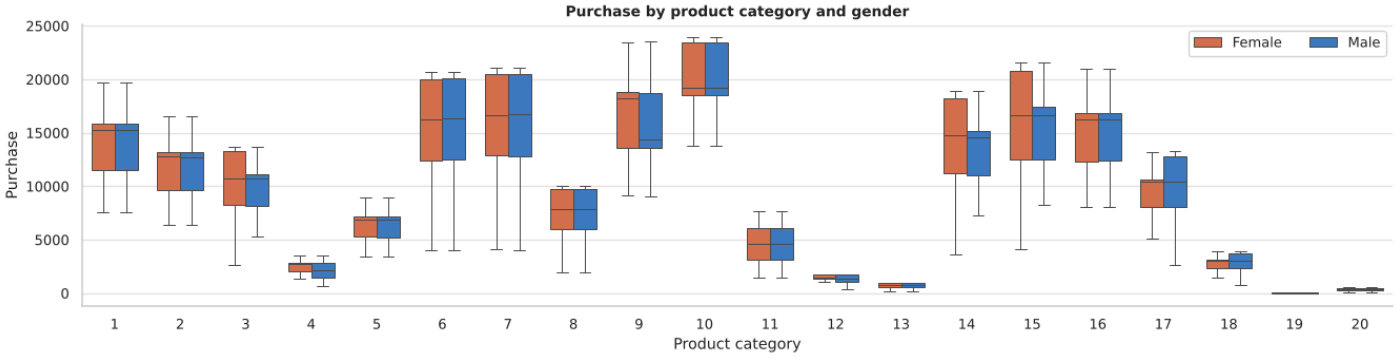
5.3 Bivariate: purchase amount across categories (box plots)

```
cols = ["Gender", "Life_Stage", "Marital_Status", "City_Category", "Stay_In_Current_City_Years",
"Occupation"]
fig, axes = plt.subplots(2, 3, figsize=(15, 7.5))
for ax, col in zip(axes.flat, cols):
    pal = {"Gender": GENDER_PAL, "Marital_Status": MARITAL_PAL, "Life_Stage": AGE_PAL}.get(col)
    kw = dict(palette=pal) if pal else dict(color=BLUE)
    sns.boxplot(data=df, x=col, y="Purchase", ax=ax, showfliers=False, width=0.55,
                showmeans=True, meanprops=dict(marker="D", markerfacecolor="white",
markeredgecolor="#0b0b0b", markersize=5), **kw)
    ax.set_title(f"Purchase by {col.replace('_', ' ')}"); ax.set_xlabel("")
```

```
    if col == "Occupation": ax.tick_params(axis="x", labelsize=8)
plt.tight_layout(); save(fig, "05_boxplots_bivariate"); plt.show()
```




```
fig, ax = plt.subplots(figsize=(15, 4))
sns.boxplot(data=df, x="Product_Category", y="Purchase", hue="Gender", palette=GENDER_PAL,
            showfliers=False, ax=ax, width=0.7, linewidth=0.8)
ax.set_title("Purchase by product category and gender"); ax.set_xlabel("Product category")
ax.legend(title="", loc="upper right", ncol=2)
plt.tight_layout(); save(fig, "06_category_gender_box"); plt.show()
```



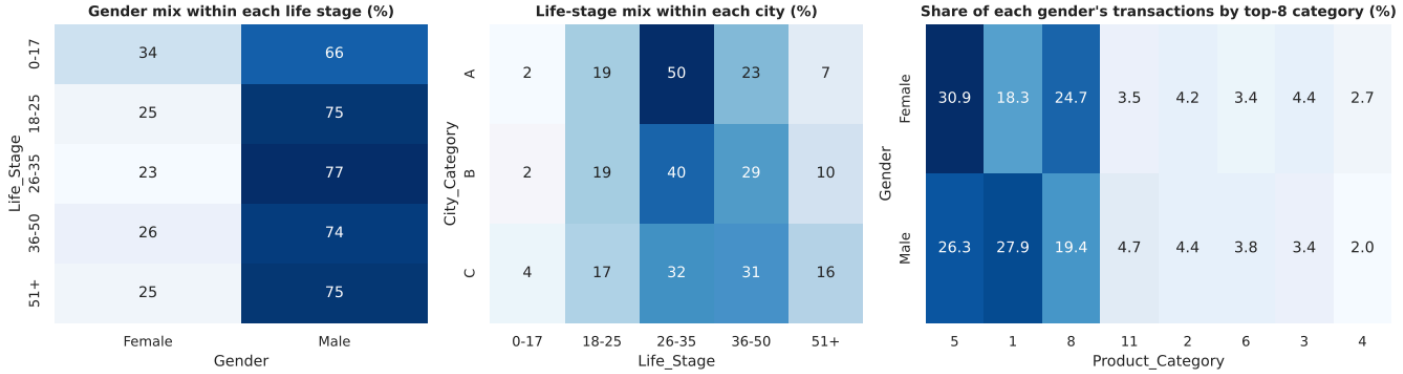
Observations (bivariate)

- **Gender:** the male box sits higher (median ~8.1K vs ~7.9K; mean 9.4K vs 8.7K) and has a longer upper whisker.
- **Life stage:** medians are close, but the mean edges up with age; 0-17 is the lowest and 51+ the highest.
- **Marital status:** the two boxes are virtually identical. Marriage does not change how much is spent per item.
- **City: City C has the highest per-transaction spend (~9.7K)** despite fewer transactions than City B; City A is the lowest (~8.9K).
- **Stay in city:** no meaningful difference across tenure.
- **Occupation:** small variation (about 8.6K-9.8K); no occupation stands out strongly.
- **Product category is the dominant driver of purchase amount**, far more than any demographic. Within a given category, male and female boxes look very similar. This is the first clue that the gender gap is about *what* people buy.

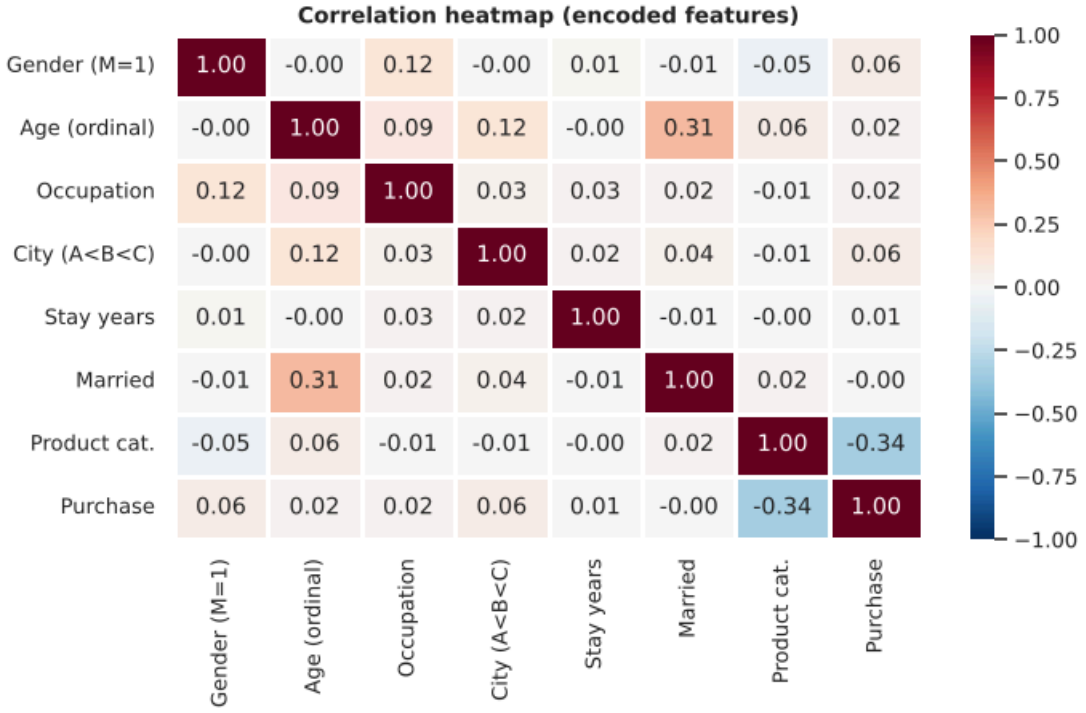
5.4 Relationships between variables: crosstabs, heatmaps and pairplot

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2), gridspec_kw={"width_ratios": [1, 1, 1.25]})
ct1 = pd.crosstab(df["Life_Stage"], df["Gender"], normalize="index") * 100
sns.heatmap(ct1, annot=True, fmt=".0f", cmap="Blues", cbar=False, ax=axes[0])
axes[0].set_title("Gender mix within each life stage (%)")
ct2 = pd.crosstab(df["City_Category"], df["Life_Stage"], normalize="index") * 100
sns.heatmap(ct2, annot=True, fmt=".0f", cmap="Blues", cbar=False, ax=axes[1])
axes[1].set_title("Life-stage mix within each city (%)")
top = df["Product_Category"].value_counts().index[:8]
```

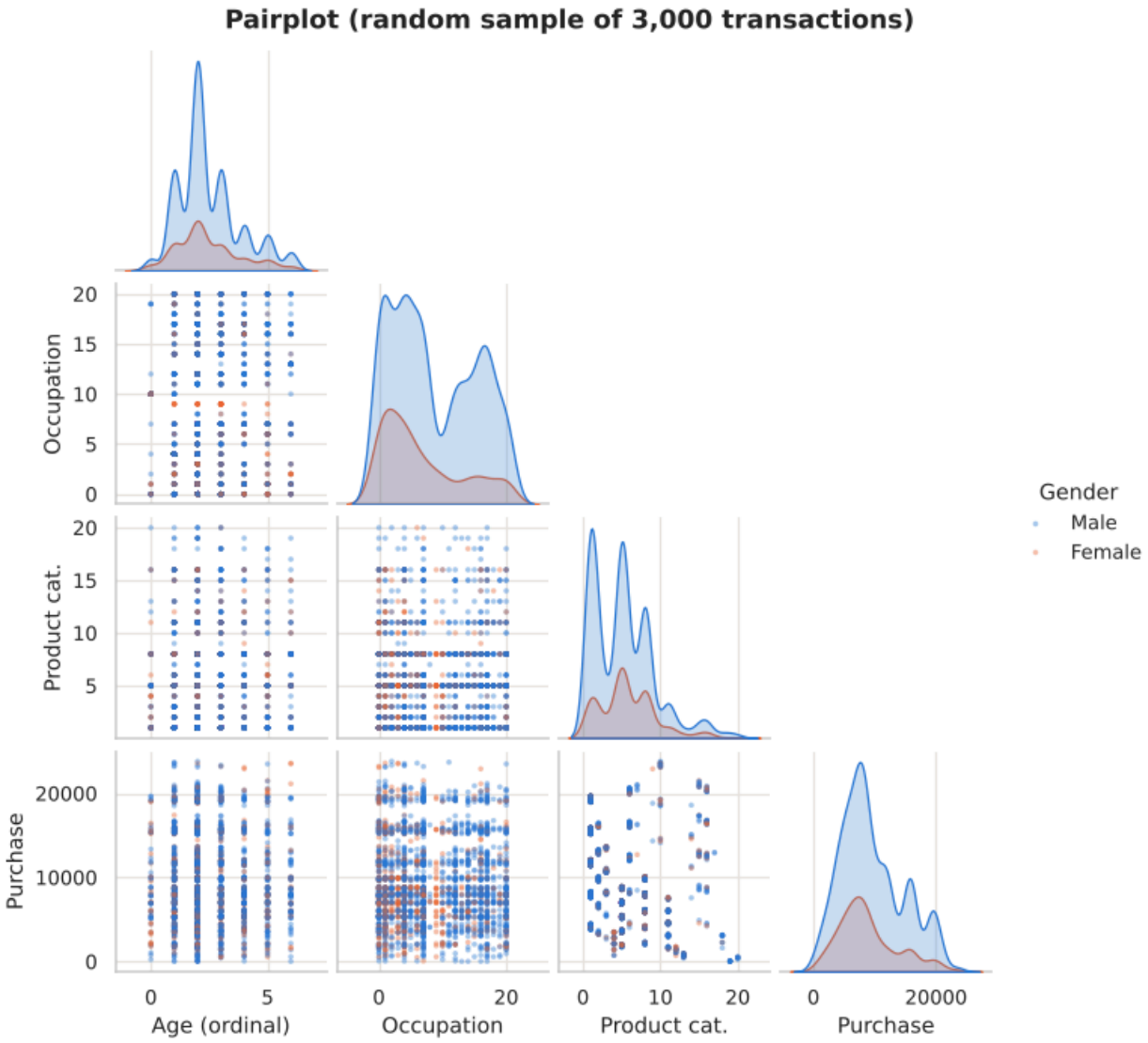
```
ct3 = (pd.crosstab(df["Product_Category"], df["Gender"], normalize="columns") * 100).loc[top]
sns.heatmap(ct3.T, annot=True, fmt=".1f", cmap="Blues", cbar=False, ax=axes[2])
axes[2].set_title("Share of each gender's transactions by top-8 category (%)")
plt.tight_layout(); save(fig, "07_crosstab_heatmaps"); plt.show()
```




```
# Correlation: encode categoricals as ordinal numbers (ordering is natural for age, city tenure)
enc = pd.DataFrame({
    "Gender (M=1)": (df["Gender"] == "Male").astype(int),
    "Age (ordinal)": df["Age"].cat.codes,
    "Occupation": df["Occupation"].astype(int),
    "City (A<B<C)": df["City_Category"].cat.codes,
    "Stay years": df["Stay_In_Current_City_Years"].astype(str).str.replace("+", "", regex=False).astype(int),
    "Married": (df["Marital_Status"] == "Married").astype(int),
    "Product cat.": df["Product_Category"].astype(int),
    "Purchase": df["Purchase"]})
fig, ax = plt.subplots(figsize=(8, 5.2))
sns.heatmap(enc.corr(), annot=True, fmt=".2f", cmap="RdBu_r", center=0, vmin=-1, vmax=1, ax=ax, linewidths=1,
linecolor="white")
ax.set_title("Correlation heatmap (encoded features)")
plt.tight_layout(); save(fig, "08_correlation_heatmap"); plt.show()
```




```
pair_df = enc.sample(3000, random_state=42).assign(Gender=lambda d: d["Gender (M=1)"].map({1: "Male", 0:
"Female"}))
g = sns.pairplot(pair_df[["Age (ordinal)", "Occupation", "Product cat.", "Purchase", "Gender"]],
hue="Gender",
                 palette=GENDER_PAL, plot_kws=dict(s=8, alpha=0.4, linewidth=0), height=1.9, corner=True)
g.fig.suptitle("Pairplot (random sample of 3,000 transactions)", y=1.02, fontweight="bold")
save(g.fig, "09_pairplot"); plt.show()
```



Observations (relationships)

- **Women are a minority in every life stage** (roughly 23-34% of transactions); the 0-17 group has the highest female share (34%).
- **City C skews older** (more 36-50 and 51+), while **City A and B skew to 26-35**. City C's higher ticket size is consistent with older shoppers spending more per item.
- **Product mix differs by gender:** Category 1 (the high-ticket staple) is **~28% of male transactions vs ~18% of female**, while the lower-ticket Categories 5 and 8 make up **~56% of female transactions vs ~46% of male**.
- **Correlation heatmap:** the only meaningful relationship with Purchase is **Product Category (r ≈ -0.34)**. Gender (r ≈ +0.06), age, city and marital status each have a very weak linear relationship with spend. Age and Marital Status are moderately correlated with each other (+0.31), as expected.
- **Pairplot:** Purchase forms horizontal bands by product category; no demographic variable shows a strong linear trend. Demographics shift the **average** a little, which is precisely what confidence intervals are designed to detect.

6. Gender: are women spending more per transaction than men?

We track every transaction made by female and male customers and compute the average amount spent per transaction.

```
def describe_groups(col):
    g = df.groupby(col, observed=True)["Purchase"]
    out = g.agg(transactions="size", mean="mean", median="median", std="std", total="sum")
    out["share of revenue %"] = out["total"] / out["total"].sum() * 100
    out["total"] = out["total"] / 1e6
    return out.rename(columns={"total": "revenue (M)"})

gender_stats = describe_groups("Gender")
gender_stats
```

Gender	transactions	mean	median	std	revenue (M)	share of revenue %
Female	135809	8,734.57	7,914.00	4,767.23	1,186.23	23.28
Male	414259	9,437.53	8,098.00	5,092.19	3,909.58	76.72

```
f, m = (df.loc[df["Gender"] == g, "Purchase"] for g in ["Female", "Male"])
gap = m.mean() - f.mean()
t, pval = stats.ttest_ind(m, f, equal_var=False)
cohen_d = gap / np.sqrt((m.var() + f.var()) / 2)
print(f"Male mean - Female mean = {gap:,.1f}  ({gap / f.mean():.1%} higher for men)")
print(f"Welch t-test: t = {t:.1f}, p-value = {pval:.2e}")
print(f"Effect size (Cohen's d) = {cohen_d:.2f}  -> statistically clear, practically modest")
```

```
Male mean - Female mean = 703.0  (8.0% higher for men)
Welch t-test: t = 46.4, p-value = 0.00e+00
Effect size (Cohen's d) = 0.14  -> statistically clear, practically modest
```

Why do men spend more per transaction? A product-mix check. If the gap is caused by *what* each gender buys, then re-weighting women's category prices by men's category mix should close most of it.

```
cat_mean = df.pivot_table(index="Product_Category", columns="Gender", values="Purchase", aggfunc="mean",
observed=True)
cat_mix  = pd.crosstab(df["Product_Category"], df["Gender"], normalize="columns")

female_actual       = (cat_mix["Female"] * cat_mean["Female"]).sum()
female_with_men_mix = (cat_mix["Male"]   * cat_mean["Female"]).sum()   # women's prices, men's basket
male_actual         = (cat_mix["Male"]   * cat_mean["Male"]).sum()

decomp = pd.Series({"Female (actual)": female_actual,
                    "Female prices x Male basket mix": female_with_men_mix,
                    "Male (actual)": male_actual})
print(decomp.round(1).to_string())
explained = (female_with_men_mix - female_actual) / (male_actual - female_actual)
print(f"\nShare of the gender gap explained by product mix: {explained:.0%}")
```

```
Female (actual)                   8,734.60
Female prices x Male basket mix   9,466.40
Male (actual)                     9,437.50

Share of the gender gap explained by product mix: 104%
```

Answer: No, women are not spending more per transaction than men.

- The average **female** transaction is **8,734.6** and the average **male** transaction is **9,437.5**, so **men spend about 703 (≈ 8%) more per transaction**.
- The difference is **statistically significant** (Welch t ≈ 46, p ≈ 0), but the effect size is **small (Cohen's d ≈ 0.14)**: it is a real but modest shift, not two different kinds of shopper.
- **Why?** The product-mix check shows that most of the gap comes from **what** men and women buy. Men put a larger share of their basket into high-ticket Category 1 (avg 13.6K), while women lean towards Categories 5 and 8 (avg 6.2K and 7.5K). Within the same category, women and men pay almost exactly the same (e.g. Category 1: 13,597 vs 13,608; Category 8: 7,500 vs 7,499). If women kept their own category prices but bought men's basket mix, their average would be **~9,466**, slightly *above* the male average: product mix explains **the entire gap**.
- Men also make **more transactions per person** and form **72% of customers**, so they generate **77% of Black Friday revenue**.

7. Gender: Central Limit Theorem and confidence intervals

Method.

1. **Bootstrap (CLT experiment):** for each group, draw a random sample of size *n* (with replacement), compute its mean, and repeat **1,000 times**. The 1,000 means form the *sampling distribution of the mean*.
2. We repeat this for **n = 300, 3,000 and 30,000** to see how sample size affects the spread.
3. The **CI** is the central 90% / 95% / 99% of that sampling distribution (percentile method). We cross-check with the analytical CLT formula **mean ± z × σ/√n** on the full data.

```
SAMPLE_SIZES = [300, 3_000, 30_000]
CONF_LEVELS  = [0.90, 0.95, 0.99]
N_BOOT = 1_000
```

```
def bootstrap_means(values, n, reps=N_BOOT, rng=RNG):
    values = np.asarray(values)
    idx = rng.integers(0, len(values), size=(reps, n))
    return values[idx].mean(axis=1)

def ci_table(col, sizes=SAMPLE_SIZES, levels=CONF_LEVELS):
    rows, boot = [], {}
    for grp, vals in df.groupby(col, observed=True)["Purchase"]:
        for n in sizes:
            means = bootstrap_means(vals.values, n)
            boot[(grp, n)] = means
            for cl in levels:
                lo, hi = np.percentile(means, [(1 - cl) / 2 * 100, (1 + cl) / 2 * 100])
                rows.append(dict(group=grp, sample_size=n, confidence=f"{cl:.0%}",
                                 mean_of_means=means.mean(), lower=lo, upper=hi, width=hi - lo))
    return pd.DataFrame(rows), boot

def clt_ci_full(col, levels=CONF_LEVELS):
    rows = []
    for grp, vals in df.groupby(col, observed=True)["Purchase"]:
        se = vals.std(ddof=1) / np.sqrt(len(vals))
        for cl in levels:
            z = stats.norm.ppf((1 + cl) / 2)
            rows.append(dict(group=grp, n=len(vals), confidence=f"{cl:.0%}", mean=vals.mean(),
                             lower=vals.mean() - z * se, upper=vals.mean() + z * se, width=2 * z * se))
    return pd.DataFrame(rows)

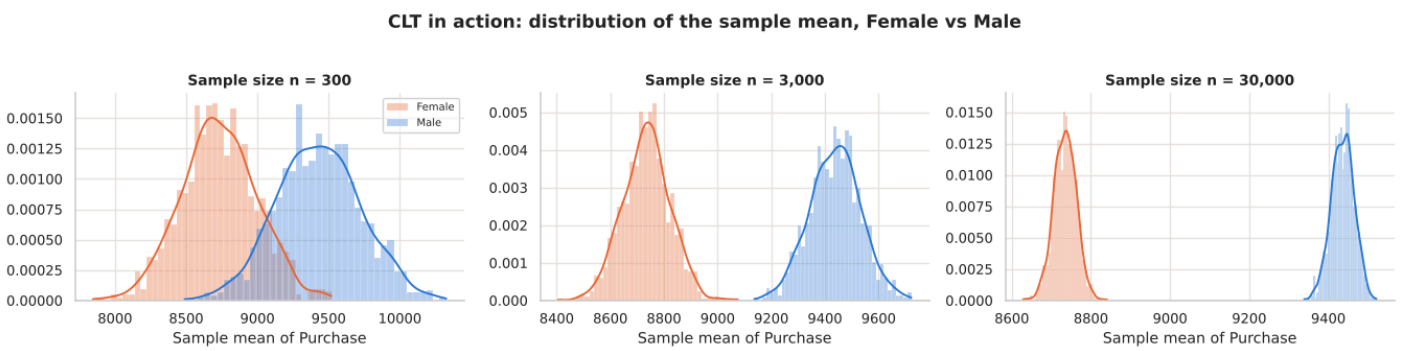
def overlap_report(ci_df, size_col="sample_size"):
    out = []
    keys = [size_col, "confidence"] if size_col else ["confidence"]
    for key, sub in ci_df.groupby(keys, sort=False):
        sub = sub.set_index("group")
        grps = list(sub.index)
        for i in range(len(grps) - 1):
            a, b = grps[i], grps[i + 1]
            ov = not (sub.loc[a, "upper"] < sub.loc[b, "lower"] or sub.loc[b, "upper"] < sub.loc[a, "lower"])
            out.append(dict(zip(keys, key if isinstance(key, tuple) else (key,))) | {"pair": f"{a} vs {b}",
"overlap": "Yes" if ov else "No"})
    return pd.DataFrame(out)
```

```
def plot_clt(boot, groups, palette, title, fname):
    fig, axes = plt.subplots(1, len(SAMPLE_SIZES), figsize=(15, 3.6))
    for ax, n in zip(axes, SAMPLE_SIZES):
        for g in groups:
            sns.histplot(boot[(g, n)], bins=40, kde=True, stat="density", color=palette[g], alpha=0.35,
                         edgecolor="white", linewidth=0.2, ax=ax, label=g)
        ax.set_title(f"Sample size n = {n:,}"); ax.set_xlabel("Sample mean of Purchase"); ax.set_ylabel("")
    axes[0].legend(fontsize=8)
    fig.suptitle(title, fontweight="bold", y=1.03)
    plt.tight_layout(); save(fig, fname); plt.show()

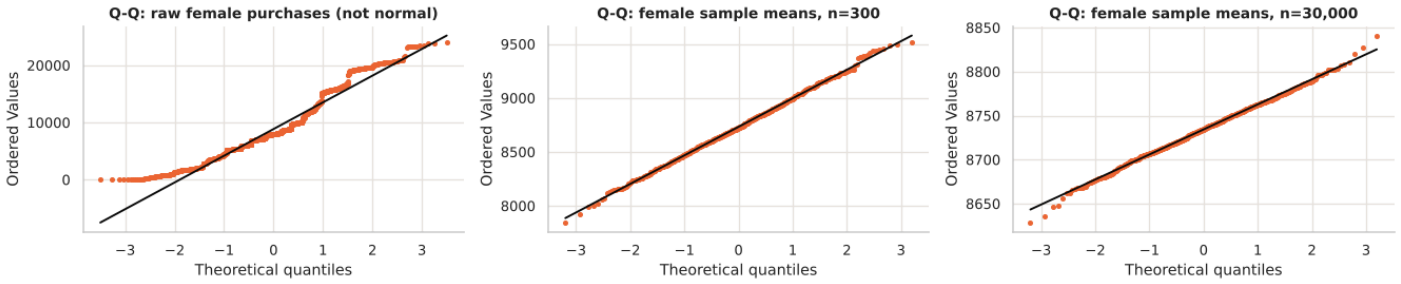
def plot_ci(ci_df, groups, palette, title, fname, size_col="sample_size"):
    sizes = ci_df[size_col].unique() if size_col else [None]
    fig, axes = plt.subplots(1, len(sizes), figsize=(15, 0.55 * len(groups) * 3 + 1), sharey=True,
squeeze=False)
    for ax, n in zip(axes.flat, sizes):
        sub = ci_df if n is None else ci_df[ci_df[size_col] == n]
        ypos = 0
        ticks, labels = [], []
        for g in groups:
            for cl in ["90%", "95%", "99%"]:
                r = sub[(sub["group"] == g) & (sub["confidence"] == cl)].iloc[0]
                centre = (r["lower"] + r["upper"]) / 2
                ax.errorbar(centre, ypos, xerr=[[centre - r["lower"]], [r["upper"] - centre]], fmt="o",
                            color=palette[g], capsize=4, lw=2, ms=5)
                ticks.append(ypos); labels.append(f"{g} {cl}"); ypos += 1
            ypos += 0.8
        ax.set_yticks(ticks); ax.set_yticklabels(labels, fontsize=8); ax.invert_yaxis()
        ax.set_title("Full data (analytical CLT)" if n is None else f"n = {n:,}")
        ax.set_xlabel("Average purchase per transaction")
    fig.suptitle(title, fontweight="bold", y=1.02)
    plt.tight_layout(); save(fig, fname); plt.show()
```

```
GENDERS = ["Female", "Male"]
gender_ci, gender_boot = ci_table("Gender")
```

```
plot_clt(gender_boot, GENDERS, GENDER_PAL, "CLT in action: distribution of the sample mean, Female vs Male",
"10_clt_gender")
```



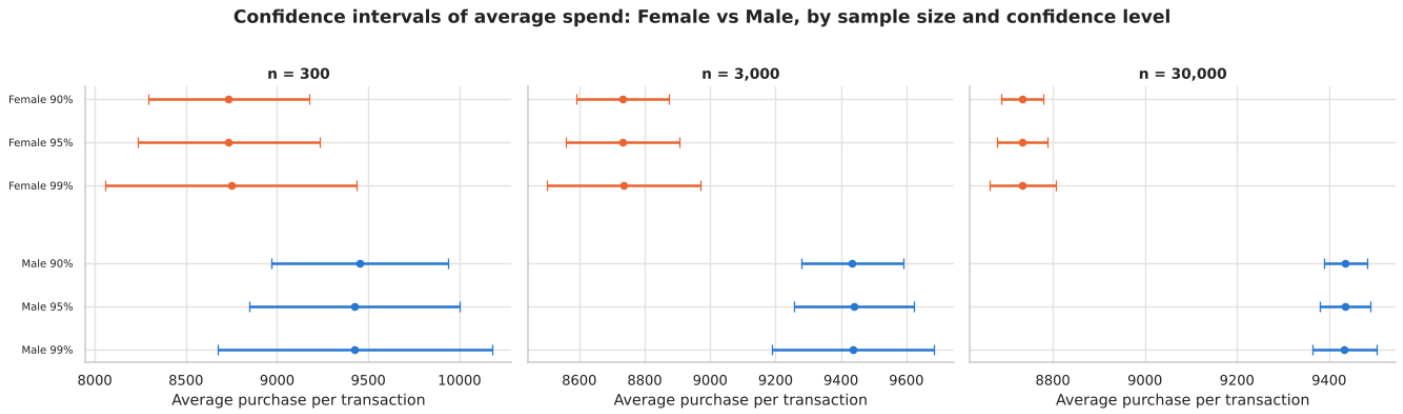

```
# Normality check: are the bootstrap means normal even though raw purchases are not?
fig, axes = plt.subplots(1, 3, figsize=(15, 3.2))
stats.probplot(df.loc[df["Gender"] == "Female", "Purchase"].sample(3000, random_state=1), dist="norm",
plot=axes[0])
axes[0].set_title("Q-Q: raw female purchases (not normal)")
stats.probplot(gender_boot[("Female", 300)], dist="norm", plot=axes[1]);  axes[1].set_title("Q-Q: female
sample means, n=300")
stats.probplot(gender_boot[("Female", 30000)], dist="norm", plot=axes[2]); axes[2].set_title("Q-Q: female
sample means, n=30,000")
for ax in axes:
    ax.get_lines()[0].set(markerfacecolor=ORANGE, markeredgecolor=ORANGE, markersize=3); ax.get_lines()
[1].set(color="#0b0b0b")
plt.tight_layout(); save(fig, "11_qq_normality"); plt.show()
```




```
gender_ci.pivot_table(index=["sample_size", "confidence"], columns="group", values=["lower", "upper",
"width"]).round(1)
```

		lower		upper		width	
	group	Female	Male	Female	Male	Female	Male
sample_size	confidence						
300	90%	8,291.70	8,968.30	9,175.60	9,936.80	883.90	968.60
	95%	8,235.20	8,849.80	9,232.90	10,001.80	997.60	1,152.00
	99%	8,056.20	8,673.70	9,437.30	10,179.70	1,381.10	1,506.00
3000	90%	8,591.90	9,278.00	8,874.40	9,591.80	282.50	313.80
	95%	8,559.30	9,255.70	8,907.00	9,624.10	347.70	368.40
	99%	8,501.70	9,189.90	8,969.20	9,685.40	467.50	495.50
30000	90%	8,686.90	9,388.00	8,779.80	9,483.00	92.90	95.00
	95%	8,678.30	9,380.20	8,787.50	9,489.20	109.20	108.90
	99%	8,661.80	9,363.10	8,806.80	9,502.90	145.00	139.70

```
plot_ci(gender_ci, GENDERS, GENDER_PAL, "Confidence intervals of average spend: Female vs Male, by sample
size and confidence level", "12_ci_gender")
overlap_report(gender_ci).pivot(index="sample_size", columns="confidence", values="overlap")
```



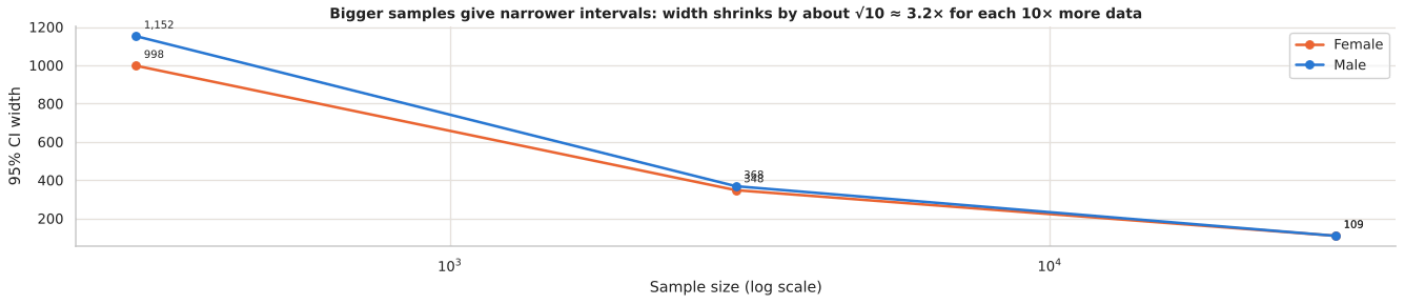
confidence	90%	95%	99%
sample_size			
300	Yes	Yes	Yes
3000	No	No	No
30000	No	No	No

```
gender_full = clt_ci_full("Gender")
display(gender_full.round(1))
overlap_report(gender_full, size_col=None)
```

	group	n	confidence	mean	lower	upper	width
0	Female	135809	90%	8,734.60	8,713.30	8,755.80	42.60
1	Female	135809	95%	8,734.60	8,709.20	8,759.90	50.70
2	Female	135809	99%	8,734.60	8,701.20	8,767.90	66.60
3	Male	414259	90%	9,437.50	9,424.50	9,450.50	26.00
4	Male	414259	95%	9,437.50	9,422.00	9,453.00	31.00
5	Male	414259	99%	9,437.50	9,417.10	9,457.90	40.80

	confidence	pair	overlap
0	90%	Female vs Male	No
1	95%	Female vs Male	No
2	99%	Female vs Male	No

```
# How CI width shrinks with sample size (95%)
fig, ax = plt.subplots(figsize=(15, 3.4))
w = gender_ci[gender_ci["confidence"] == "95%"]
for g in GENDERS:
    s = w[w["group"] == g]
    ax.plot(s["sample_size"], s["width"], marker="o", lw=2, color=GENDER_PAL[g], label=g)
    for _, r in s.iterrows():
        ax.annotate(f"{r['width']:,.0f}", (r["sample_size"], r["width"]), textcoords="offset points", xytext=
(6, 6), fontsize=8)
ax.set_xscale("log"); ax.set_xlabel("Sample size (log scale)"); ax.set_ylabel("95% CI width")
ax.set_title("Bigger samples give narrower intervals: width shrinks by about √10 ≈ 3.2× for each 10× more
data")
ax.legend(); plt.tight_layout(); save(fig, "13_ci_width_vs_n"); plt.show()
```



Observations: CLT and confidence intervals for gender

- **The CLT holds:** raw purchases are skewed and multi-modal, yet the bootstrap means form a clean **bell curve** at every sample size (Q-Q points sit on the line). The centre of each bell curve matches the group mean.
- **Sample size controls precision.** As *n* goes from 300 to 3,000 to 30,000, the spread of the sample means shrinks by about √10 each time, and the **95% CI narrows from ~1,000-1,150 to ~350-370 to ~110**.
- **Confidence level controls width.** For a given *n*, the 99% CI is the widest and the 90% CI the narrowest. Higher confidence costs precision.
- **Overlap:**
  - At **n = 300** the male and female intervals **overlap** at all three confidence levels: a small sample cannot tell the groups apart.
  - From **n = 3,000** upward the intervals **do not overlap**, even at 99%.
  - Using **all the data** (analytical CLT), the 95% intervals are **Female [8,709, 8,760]** and **Male [9,422, 9,453]**, with a gap of more than 650 between them.
- **Conclusion for the 50M + 50M population:** we can be 95% (and even 99%) confident that the **average male transaction is higher than the average female transaction**. Women do **not** spend more per transaction.

How Walmart can use this

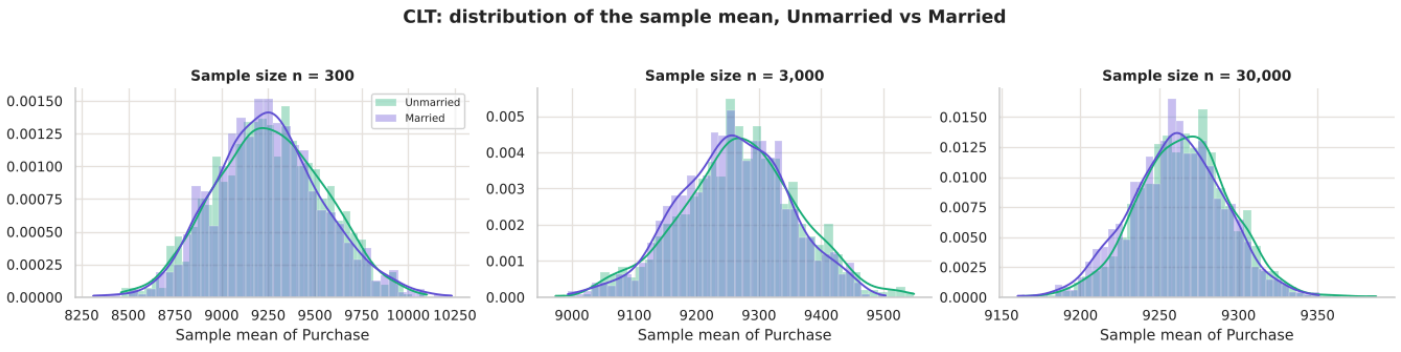
- Because the intervals do not overlap, the difference is **real and repeatable**, so gender-specific strategies are justified.
- Since the gap comes mostly from **basket mix**, the highest-value action is to **lift women's spend in high-ticket categories** (e.g. Category 1, 6, 10) through targeted bundles, curated "Black Friday picks" and personalised offers.
- The female customer base is **under-represented (28% of shoppers)** even though the population is 50/50: there is a large **acquisition opportunity**.

8. Marital status: Married vs Unmarried

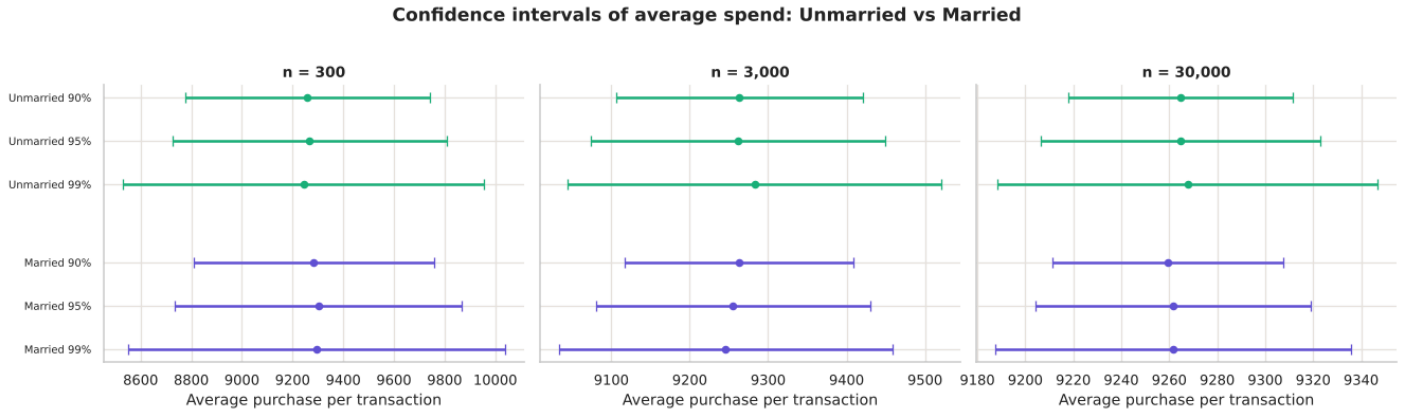
```
describe_groups("Marital_Status")
```

Marital_Status	transactions	mean	median	std	revenue (M)	share of revenue %
Unmarried	324731	9,265.91	8,044.00	5,027.35	3,008.93	59.05
Married	225337	9,261.17	8,051.00	5,016.90	2,086.89	40.95

```
MARITAL = ["Unmarried", "Married"]
marital_ci, marital_boot = ci_table("Marital_Status")
plot_clt(marital_boot, MARITAL, MARITAL_PAL, "CLT: distribution of the sample mean, Unmarried vs Married",
"14_clt_marital")
```




```
plot_ci(marital_ci, MARITAL, MARITAL_PAL, "Confidence intervals of average spend: Unmarried vs Married",
"15_ci_marital")
marital_full = clt_ci_full("Marital_Status")
display(marital_full.round(1))
display(overlap_report(marital_ci).pivot(index="sample_size", columns="confidence", values="overlap"))
u, mr = (df.loc[df["Marital_Status"] == g, "Purchase"] for g in MARITAL)
print(f"Welch t-test: p-value = {stats.ttest_ind(u, mr, equal_var=False).pvalue:.3f}")
```



	group	n	confidence	mean	lower	upper	width
0	Unmarried	324731	90%	9,265.90	9,251.40	9,280.40	29.00
1	Unmarried	324731	95%	9,265.90	9,248.60	9,283.20	34.60
2	Unmarried	324731	99%	9,265.90	9,243.20	9,288.60	45.40
3	Married	225337	90%	9,261.20	9,243.80	9,278.60	34.80
4	Married	225337	95%	9,261.20	9,240.50	9,281.90	41.40
5	Married	225337	99%	9,261.20	9,234.00	9,288.40	54.40

confidence	90%	95%	99%
sample_size			
300	Yes	Yes	Yes
3000	Yes	Yes	Yes
30000	Yes	Yes	Yes

```
Welch t-test: p-value = 0.731
```

Observations: Married vs Unmarried

- Average spend per transaction is **9,265.9 (Unmarried)** vs **9,261.2 (Married)**: a difference of under 5.
- The sampling distributions **lie on top of each other** at every sample size, and the confidence intervals **overlap at every sample size and every confidence level** (full-data 95%: Unmarried [9,249, 9,283], Married [9,241, 9,282]). The t-test agrees ($p \approx 0.74$).
- **Conclusion:** for the population, **marital status has no measurable effect on spend per transaction.**
- **How Walmart can use this:** do not split Black Friday pricing or discount depth by marital status. Instead use marital status for **what** to promote (e.g. home and family bundles for married shoppers, lifestyle and tech for unmarried shoppers), since the *ticket size* is the same.

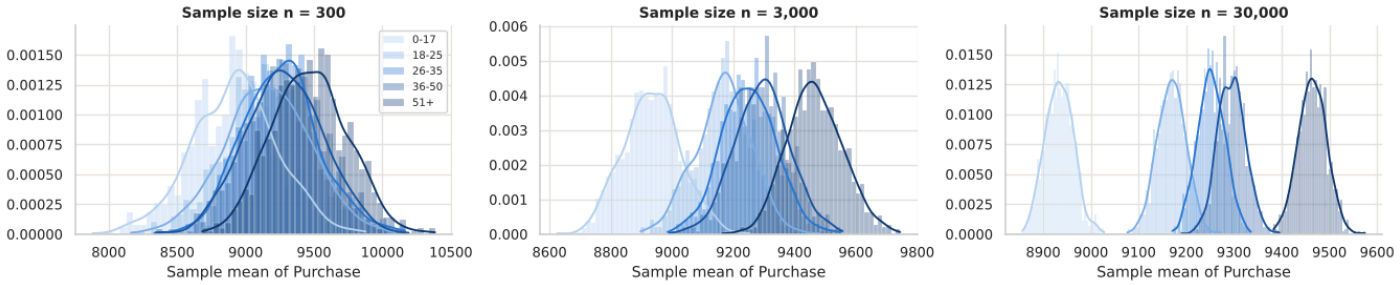
9. Age: life-stage bins (0-17, 18-25, 26-35, 36-50, 51+)

The original seven age bins are merged into the five life stages suggested in the case: 36-45 and 46-50 become **36-50**, and 51-55 and 55+ become **51+**.

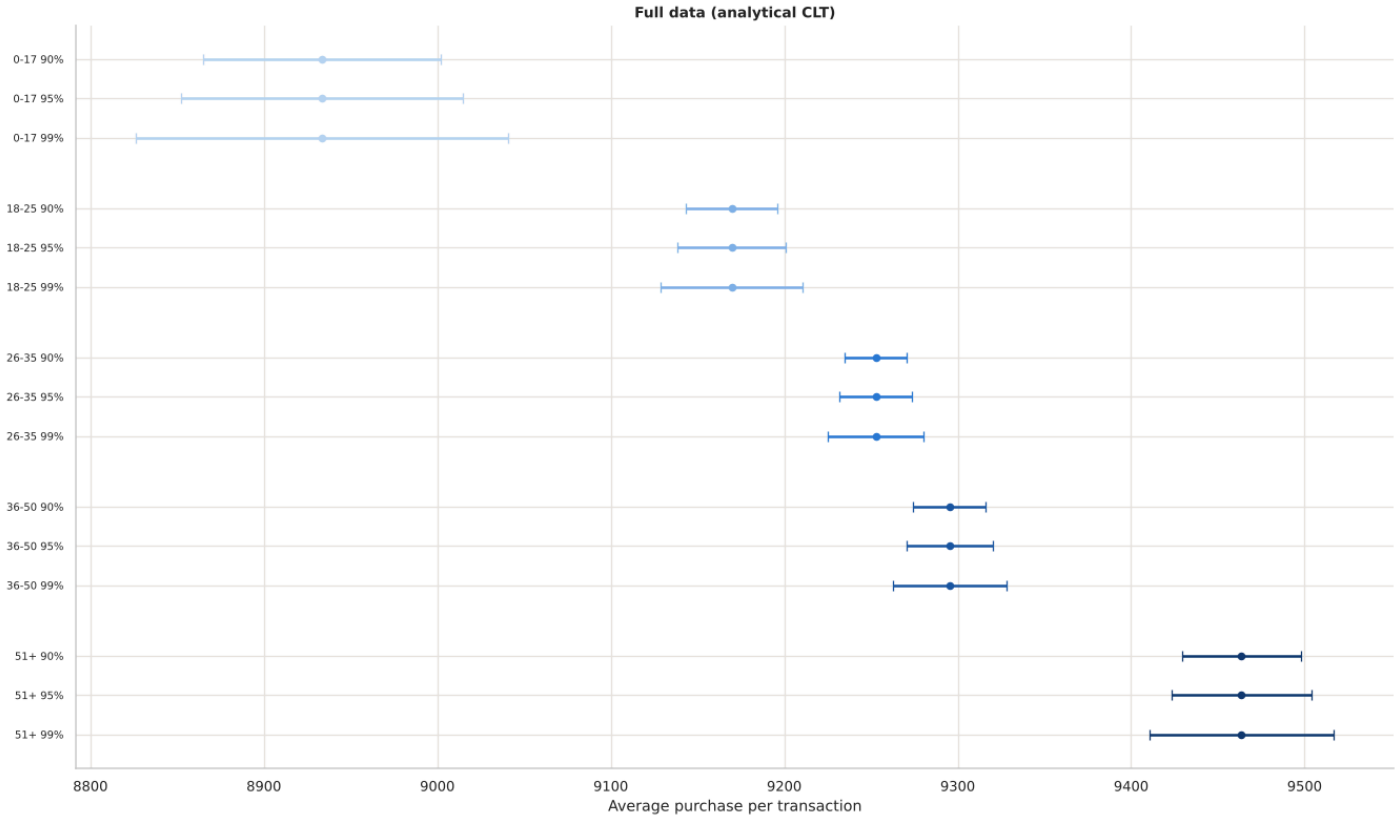
```
describe_groups("Life_Stage")
```

Life_Stage	transactions	mean	median	std	revenue (M)	share of revenue %
0-17	15102	8,933.46	7,986.00	5,111.11	134.91	2.65
18-25	99660	9,169.66	8,027.00	5,034.32	913.85	17.93
26-35	219587	9,252.69	8,030.00	5,010.53	2,031.77	39.87
36-50	155714	9,295.33	8,053.00	5,006.93	1,447.41	28.40
51+	60005	9,463.66	8,122.00	5,061.16	567.87	11.14

```
age_ci, age_boot = ci_table("Life_Stage")
plot_clt(age_boot, AGE_ORDER, AGE_PAL, "CLT: distribution of the sample mean by life stage", "16_clt_age")
```




```
age_full = clt_ci_full("Life_Stage")
plot_ci(age_full.assign(sample_size=None), AGE_ORDER, AGE_PAL,
        "Confidence intervals of average spend by life stage (full data, analytical CLT)", "17_ci_age",
size_col=None)
display(age_full.round(1))
overlap_report(age_full, size_col=None).pivot(index="pair", columns="confidence", values="overlap").loc[[f"
{a} vs {b}" for a, b in zip(AGE_ORDER, AGE_ORDER[1:])]]
```



	group	n	confidence	mean	lower	upper	width
0	0-17	15102	90%	8,933.50	8,865.10	9,001.90	136.80
1	0-17	15102	95%	8,933.50	8,851.90	9,015.00	163.00
2	0-17	15102	99%	8,933.50	8,826.30	9,040.60	214.30
3	18-25	99660	90%	9,169.70	9,143.40	9,195.90	52.50
4	18-25	99660	95%	9,169.70	9,138.40	9,200.90	62.50
5	18-25	99660	99%	9,169.70	9,128.60	9,210.70	82.20
6	26-35	219587	90%	9,252.70	9,235.10	9,270.30	35.20
7	26-35	219587	95%	9,252.70	9,231.70	9,273.60	41.90
8	26-35	219587	99%	9,252.70	9,225.10	9,280.20	55.10
9	36-50	155714	90%	9,295.30	9,274.50	9,316.20	41.70
10	36-50	155714	95%	9,295.30	9,270.50	9,320.20	49.70
11	36-50	155714	99%	9,295.30	9,262.60	9,328.00	65.40
12	51+	60005	90%	9,463.70	9,429.70	9,497.60	68.00
13	51+	60005	95%	9,463.70	9,423.20	9,504.20	81.00
14	51+	60005	99%	9,463.70	9,410.40	9,516.90	106.40

confidence	90%	95%	99%
pair			
0-17 vs 18-25	No	No	No
18-25 vs 26-35	No	No	No
26-35 vs 36-50	No	Yes	Yes
36-50 vs 51+	No	No	No

```
print("Overlap of neighbouring age groups in bootstrap CIs, by sample size:")
age_ov = overlap_report(age_ci)
age_ov.pivot_table(index=["pair"], columns=["sample_size", "confidence"], values="overlap",
aggfunc="first").loc[[f"{a} vs {b}" for a, b in zip(AGE_ORDER, AGE_ORDER[1:])]]
```

```
Overlap of neighbouring age groups in bootstrap CIs, by sample size:
```

sample_size	300			3000			30000		
confidence	90%	95%	99%	90%	95%	99%	90%	95%	99%
pair									
0-17 vs 18-25	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No
18-25 vs 26-35	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
26-35 vs 36-50	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
36-50 vs 51+	Yes	Yes	Yes	Yes	Yes	Yes	No	No	No

```
f_stat, p_anova = stats.f_oneway(*[g.values for _, g in df.groupby("Life_Stage", observed=True)["Purchase"]])
print(f"One-way ANOVA across life stages: F = {f_stat:.1f}, p-value = {p_anova:.2e}")
```

```
One-way ANOVA across life stages: F = 50.6, p-value = 1.06e-42
```

Observations: Age

- Average spend per transaction **rises with age**: 0-17 **8,933** → 18-25 **9,170** → 26-35 **9,253** → 36-50 **9,295** → 51+ **9,464**. The 51+ group spends about **530 (6%) more per item** than teenagers.
- With **small samples (n = 300 and 3,000)**, every neighbouring pair overlaps: the age effect is invisible. At **n = 30,000**, 0-17 and 51+ separate from their neighbours, while the middle groups still overlap. (Note: 0-17 has only 15,102 transactions, so its n = 30,000 bootstrap re-uses rows; the full-data CI below is the reliable one for that group.)
- With **all the data**:
  - **0-17** is clearly below everyone else, and **51+** is clearly above everyone else, at every confidence level.
  - **18-25 vs 26-35** do **not** overlap (although they come close at 99%).
  - **26-35 vs 36-50** is the borderline pair: **no overlap at 90%, but overlap at 95% and 99%**. This shows how the business's choice of confidence level can change the conclusion. These two groups are best treated as **one "prime-age" segment**.
- ANOVA confirms that age groups differ overall (p ≈ 0).

- **How Walmart can use this:** the **26-50** age band brings the most transactions and revenue (~68% of transactions), so it is the core audience. The **51+** group has the **highest ticket size**, making it ideal for premium and high-value offers. **0-17** has the lowest ticket and is best reached through family / parent-led promotions.

10. Bonus: the customer-level view (total spend per shopper)

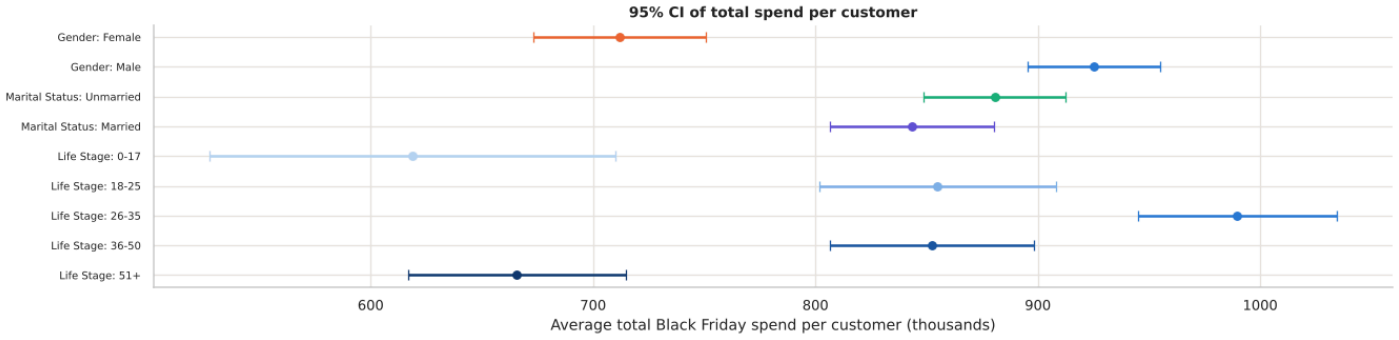
Per-transaction averages answer "how big is each purchase?". Management may also ask "how much does each **customer** spend in total on Black Friday?". Here each of the 5,891 customers is one observation.

```
def customer_ci(col, cl=0.95):
    rows = []
    for g, v in users.groupby(col, observed=True)["Total_Spend"]:
        se = v.std(ddof=1) / np.sqrt(len(v)); z = stats.norm.ppf((1 + cl) / 2)
        rows.append(dict(segment=col, group=g, customers=len(v), mean=v.mean(), lower=v.mean() - z * se,
upper=v.mean() + z * se))
    return pd.DataFrame(rows)

cust_ci = pd.concat([customer_ci(c) for c in ["Gender", "Marital_Status", "Life_Stage"]], ignore_index=True)
display(cust_ci.round(0))

fig, ax = plt.subplots(figsize=(15, 3.8))
pal = GENDER_PAL | MARITAL_PAL | AGE_PAL
for i, r in cust_ci.iterrows():
    ax.errorbar(r["mean"] / 1e3, i, xerr=[[(r["mean"] - r["lower"]) / 1e3], [(r["upper"] - r["mean"]) /
1e3]],
                fmt="o", color=pal[r["group"]], capsize=4, lw=2)
ax.set_yticks(range(len(cust_ci))); ax.set_yticklabels(cust_ci["segment"].str.replace("_", " ") + ": " +
cust_ci["group"].astype(str), fontsize=8)
ax.invert_yaxis(); ax.set_xlabel("Average total Black Friday spend per customer (thousands)")
ax.set_title("95% CI of total spend per customer")
plt.tight_layout(); save(fig, "18_customer_level_ci"); plt.show()
```

	segment	group	customers	mean	lower	upper
0	Gender	Female	1666	712,024.00	673,255.00	750,793.00
1	Gender	Male	4225	925,344.00	895,618.00	955,070.00
2	Marital_Status	Unmarried	3417	880,576.00	848,742.00	912,410.00
3	Marital_Status	Married	2474	843,527.00	806,670.00	880,384.00
4	Life_Stage	0-17	218	618,868.00	527,664.00	710,071.00
5	Life_Stage	18-25	1069	854,863.00	801,634.00	908,092.00
6	Life_Stage	26-35	2053	989,659.00	945,035.00	1,034,283.00
7	Life_Stage	36-50	1698	852,422.00	806,470.00	898,374.00
8	Life_Stage	51+	853	665,729.00	616,782.00	714,677.00



Observations: customer level

- **Gender:** an average man spends **~925K** over Black Friday vs **~712K** for a woman. The 95% CIs (**[896K, 955K]** vs **[673K, 751K]**) **do not overlap**, so the conclusion is the same as per transaction, and stronger: men buy slightly bigger tickets **and** buy more items.
- **Marital status:** intervals overlap (unmarried ~881K vs married ~844K). No significant difference.
- **Age:** the **26-35** shopper is the most valuable customer (~990K), significantly ahead of 51+ (~666K) and 0-17 (~619K). The 51+ group buys **fewer but pricier** items, while 26-35 buy **many** items.

```
summary_ci = pd.concat([clt_ci_full(c).assign(segment=c) for c in ["Gender", "Marital_Status",
"Life_Stage"]])
summary_ci = summary_ci[["segment", "group", "n", "confidence", "mean", "lower", "upper", "width"]]
os.makedirs("../results", exist_ok=True)
summary_ci.round(2).to_csv("../results/confidence_intervals_summary.csv", index=False)
gender_ci.round(2).to_csv("../results/bootstrap_ci_gender.csv", index=False)
marital_ci.round(2).to_csv("../results/bootstrap_ci_marital.csv", index=False)
age_ci.round(2).to_csv("../results/bootstrap_ci_age.csv", index=False)
cust_ci.round(2).to_csv("../results/customer_level_ci.csv", index=False)
summary_ci[summary_ci["confidence"] == "95%"].round(1).reset_index(drop=True)
```

	segment	group	n	confidence	mean	lower	upper	width
0	Gender	Female	135809	95%	8,734.60	8,709.20	8,759.90	50.70
1	Gender	Male	414259	95%	9,437.50	9,422.00	9,453.00	31.00
2	Marital_Status	Unmarried	324731	95%	9,265.90	9,248.60	9,283.20	34.60
3	Marital_Status	Married	225337	95%	9,261.20	9,240.50	9,281.90	41.40
4	Life_Stage	0-17	15102	95%	8,933.50	8,851.90	9,015.00	163.00
5	Life_Stage	18-25	99660	95%	9,169.70	9,138.40	9,200.90	62.50
6	Life_Stage	26-35	219587	95%	9,252.70	9,231.70	9,273.60	41.90
7	Life_Stage	36-50	155714	95%	9,295.30	9,270.50	9,320.20	49.70
8	Life_Stage	51+	60005	95%	9,463.70	9,423.20	9,504.20	81.00

11. Final insights

Distribution of variables and relationships

1. **Purchase is right-skewed and multi-modal** (mean 9,264 > median 8,047). The peaks come from product-category price bands, so **product category is the strongest driver of spend** (correlation ≈ -0.34). Demographics shift the average only modestly.
2. **The shopper base is skewed:** 75% of transactions and 72% of customers are male; 40% of transactions come from ages 26-35; 59% are unmarried; City B has the most transactions but **City C has the highest ticket size**.
3. **Outliers (0.49%) are real high-value purchases** from premium categories and were kept.

Generalising to the population (CLT and confidence intervals)

4. **Gender:** men spend **~703 (8%) more per transaction** than women. The CIs do not overlap at 90, 95 or 99% (full data), so this holds for the 50M + 50M population. **Women do not spend more per transaction.**
5. **Why:** the gender gap is explained entirely by **basket mix**: men buy more of high-ticket Category 1, women more of mid/low-ticket Categories 5 and 8. Within the same category, women pay the same as men.
6. **Marital status makes no difference** to spend per transaction: the CIs overlap at every sample size and confidence level.
7. **Age matters, modestly:** spend per transaction rises with age. 0-17 is clearly lowest and 51+ clearly highest; 26-35 and 36-50 are statistically almost identical (overlap at 95%/99%).
8. **Sample size matters:** with n = 300 none of the gender, marital or age differences are detectable. The gender gap appears from n = 3,000; the smaller age differences need tens of thousands of observations. Walmart should rely on large samples before making segment-level decisions.
9. **Confidence level is a business trade-off:** a 99% interval is safer but wider; 90% is tighter but riskier. The 26-35 vs 36-50 comparison flips between "different" (90%) and "same" (95%), showing that borderline differences should not drive big decisions.
10. **Customer-level view confirms it:** a male customer spends ~925K vs ~712K for a female customer over Black Friday, and the 26-35 shopper is the most valuable customer overall.

12. Recommendations for Walmart

Simple, actionable steps, no jargon.

1. **Win more women shoppers.** Women are half the population but only about a quarter of Black Friday shoppers. Run women-focused Black Friday campaigns, early-access sales and referral rewards to bring more of them into stores.
2. **Help women discover bigger-ticket products.** Women already spend well, but in lower-price categories. Create "Top Black Friday picks" bundles that pair their favourite categories (5 and 8) with higher-value items from Categories 1, 6 and 10, and offer "spend more, save more" deals.

3. **Keep male shoppers loyal.** Men bring 77% of Black Friday revenue. Protect it with loyalty points, member-only deals and stock guarantees on the high-ticket items they buy most.
4. **Do not change prices based on marital status.** Married and unmarried customers spend the same per item. Use marital status only to choose *which* products to show (family and home bundles vs lifestyle and gadgets).
5. **Focus the main campaign on ages 26-50.** They make two-thirds of all purchases. Treat 26-35 and 36-50 as one group in planning.
6. **Offer premium deals to the 51+ group.** They have the highest spend per item. Promote quality, durable and premium products with easy in-store help and delivery.
7. **Reach teenagers through parents.** The 0-17 group spends the least per item. Bundle youth products into family offers instead of targeting teens directly.
8. **Invest more in City C, grow City A.** City C shoppers spend the most per item, so stock premium ranges there. City A has the lowest ticket size and could benefit from upselling and bundles.
9. **Stock and staff for the big three categories.** Categories 1, 5 and 8 make up about three in four transactions. Make sure they never run out during Black Friday.
10. **Measure before acting.** Small samples hide real differences. Test new offers on large groups (thousands of customers) and use 95% confidence as the standard for big decisions.

Data: Walmart Black Friday transactions (550,068 rows). Full code, data, images and results: see the project repository README.